

CCSAR

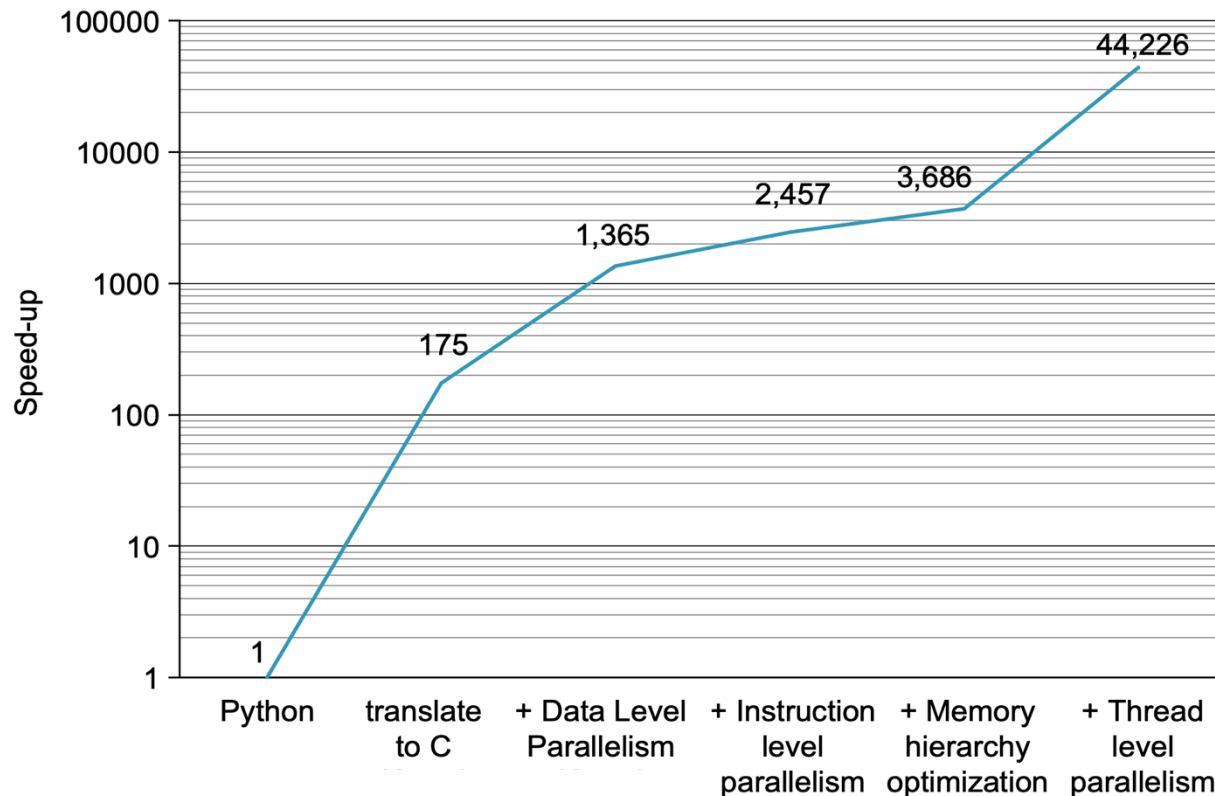
Course Overview

Arquiteturas de Computadores para Sistemas Críticos
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

The Question Behind This Course

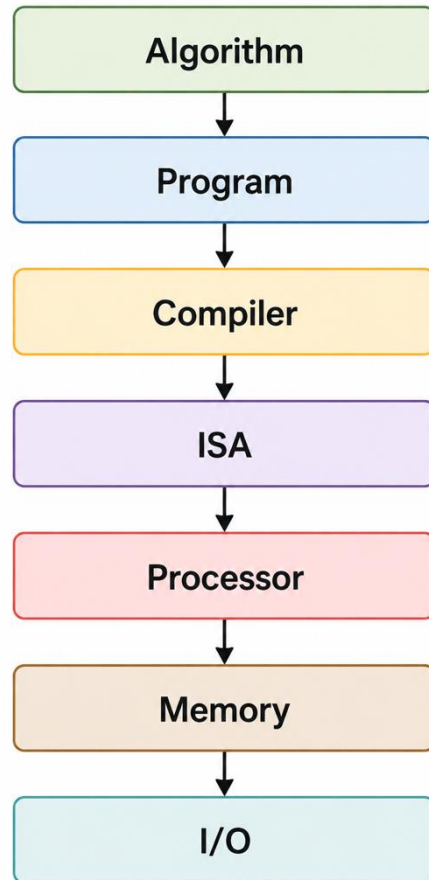
- Why does the same program **run at radically different speeds?**
 - The algorithm is the same, the execution environment is not



Where Does Performance Come From?

- Performance is a **system property**
- Performance depends on several layers:
 - **Algorithm** – How much work is required?
 - **Language + Compiler + ISA** – What instructions are executed?
 - **Processor + Memory** – How efficiently can instructions execute and data move?
 - **OS + I/O** – How efficiently can the system interact with the outside world?
- Therefore, there is no single "speed" knob

A Hierarchy of Abstractions



- **Why do these layers matter?**
- **If we change one layer, which of these four properties can be affected?**
 - **Correctness** – Does it work as intended?
 - **Performance** – How fast is it?
 - **Efficiency** – How well does it use resources?
 - **Scalability** – How well does it handle increasing demands?

What You Will Learn

- **The goal**

- Understand not only how computers work, but **why they are designed this way**

- **We will answer questions such as:**

- How are instructions represented and executed?
- How is a processor designed?
- Why do memory hierarchies work?
- How does pipelining improve performance?
- How can programs exploit parallelism?
- How do hardware and software interact?

Why Study Computer Architecture?

- **Architecture is where software meets hardware**
- **Computer architecture underpins modern computing**
 - Operating systems, compilers and runtimes
 - Embedded systems and virtualization
 - Databases and networks
 - Security and high-performance computing
- **The key idea**
 - Performance, security, and dependability often depend on what happens across abstraction boundaries

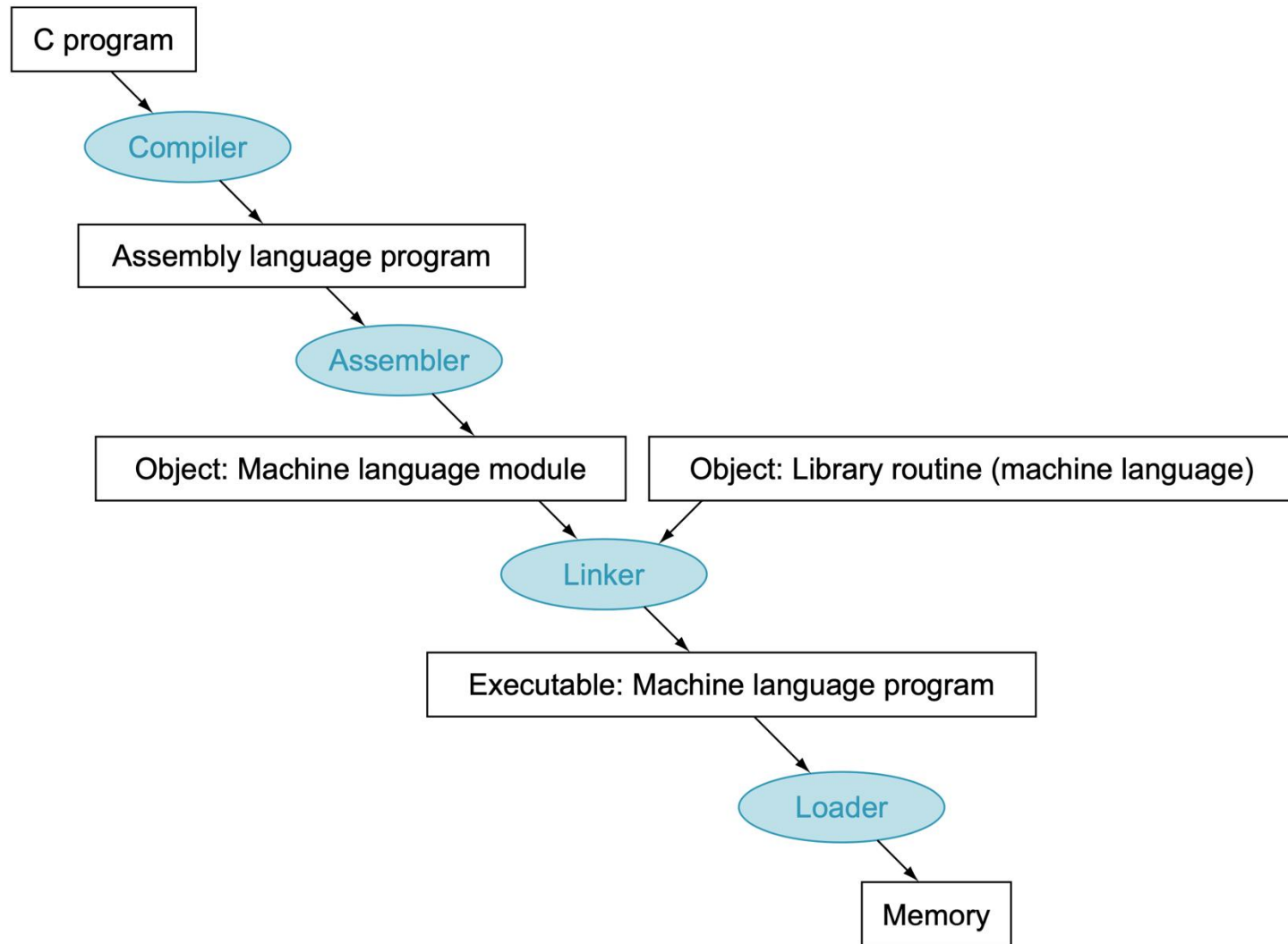
Architecture in critical systems

- **In critical systems, architecture must balance more than performance**
 - Performance — meeting timing and throughput requirements
 - Efficiency — using resources effectively
 - Dependability — providing reliable and trustworthy operation
 - Predictability — ensuring bounded and repeatable behavior
- **The key idea**
 - In critical systems, architecture must balance performance with efficiency, dependability, and predictability

Seven Great Ideas In Computing

- You will encounter all seven ideas throughout the course
 - Today we will only touch on some of them
- 1. **Abstraction** – Simplify complex systems
- 2. **Common case** – Make the common case fast
- 3. **Pipelining** – Overlap operations
- 4. **Prediction** – Speculate when useful
- 5. **Parallelism** – Perform multiple operations simultaneously
- 6. **Memory hierarchy** – Trade capacity for speed
- 7. **Redundancy** – Improve dependability

How Does a Program Reach The Processor?



Levels of Program Code

■ High-level language

- Close to the problem domain
- Productive and portable

High-level
language
program
(in C)

```
swap(size_t v[], size_t k)
{
    size_t temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

■ Assembly language

- Human-readable representation of machine instructions
- Exposes processor-level operations

Assembly
language
program
(for RISC-V)

```
swap:
    slli x6, x11, 3
    add x6, x10, x6
    lw x5, 0(x6)
    lw x7, 4(x6)
    sw x7, 0(x6)
    sw x5, 4(x6)
    jalr x0, 0(x1)
```

■ Machine-code representation

- Binary digits (bits)
- Encoded instructions and data

Binary machine
language
program
(for RISC-V)

```
00000000001101011001001100010011
00000000011001010000001100110011
00000000000000110011001010000011
00000000100000110011001110000011
00000000011100110011000000100011
00000000010100110011010000100011
00000000000000001000000011001111
```

What Does The Processor Actually See?

- A processor does not execute C code:

```
f = (g + h) - (i + j);
```

- It executes machine instructions such as:

```
add x5, x20, x21  
add x6, x22, x23  
sub x19, x5, x6
```

- So, what defines these instructions?
 - The **Instruction Set Architecture** — ISA

Instruction Set Architecture (ISA)

- **The contract between software and hardware**
 - Provides a stable programming model independent of processor implementation
 - Enables different processors to run the same software
- **An ISA defines:**
 - Available instructions
 - Registers and addressing modes
 - Memory model
 - Exception and privilege mechanisms
- **The ISA defines **what** a processor must do**
 - The microarchitecture defines **how** it does it

Why Does The ISA Matter?

- **The same ISA can be implemented in processors with very different:**
 - Performance
 - Power consumption
 - Cost
 - Microarchitectures
- **Software can remain compatible**
 - The ISA is stable; implementations evolve

Real-world ISAs

- **Different ISAs, different ecosystems**

ISA	Typical systems
ARM	Mobile, embedded, servers
AMD64 / x86-64	PCs, laptops, servers
IBM z/Architecture	Enterprise mainframes
RISC-V	Embedded, controllers, accelerators, application processors

- **An ISA is more than an instruction set**

- Compatibility + Tools + Operating Systems + Applications + Ecosystem

What Makes RISC-V Different?

- **Open standard**
 - No proprietary ISA licensing required
- **Modular**
 - Small base ISA + Optional extensions
- **Designed for extensibility**
 - Different implementations can target different needs

One ISA for Everything?

- **Could a common ISA reduce fragmentation?**
- **A common ISA could potentially:**
 - Simplify software development
 - Enable common toolchains
 - Improve portability
 - Reduce dependence on proprietary architectures
 - Support different classes of processors
- **But there is a trade-off**
 - Modularity enables specialization – but too much variation can create fragmentation

What Does a Computer Actually Contain?

■ **Processor**

- Executes instructions
- Maintains execution state in registers

■ **Memory**

- Stores instructions and data

■ **Interconnect**

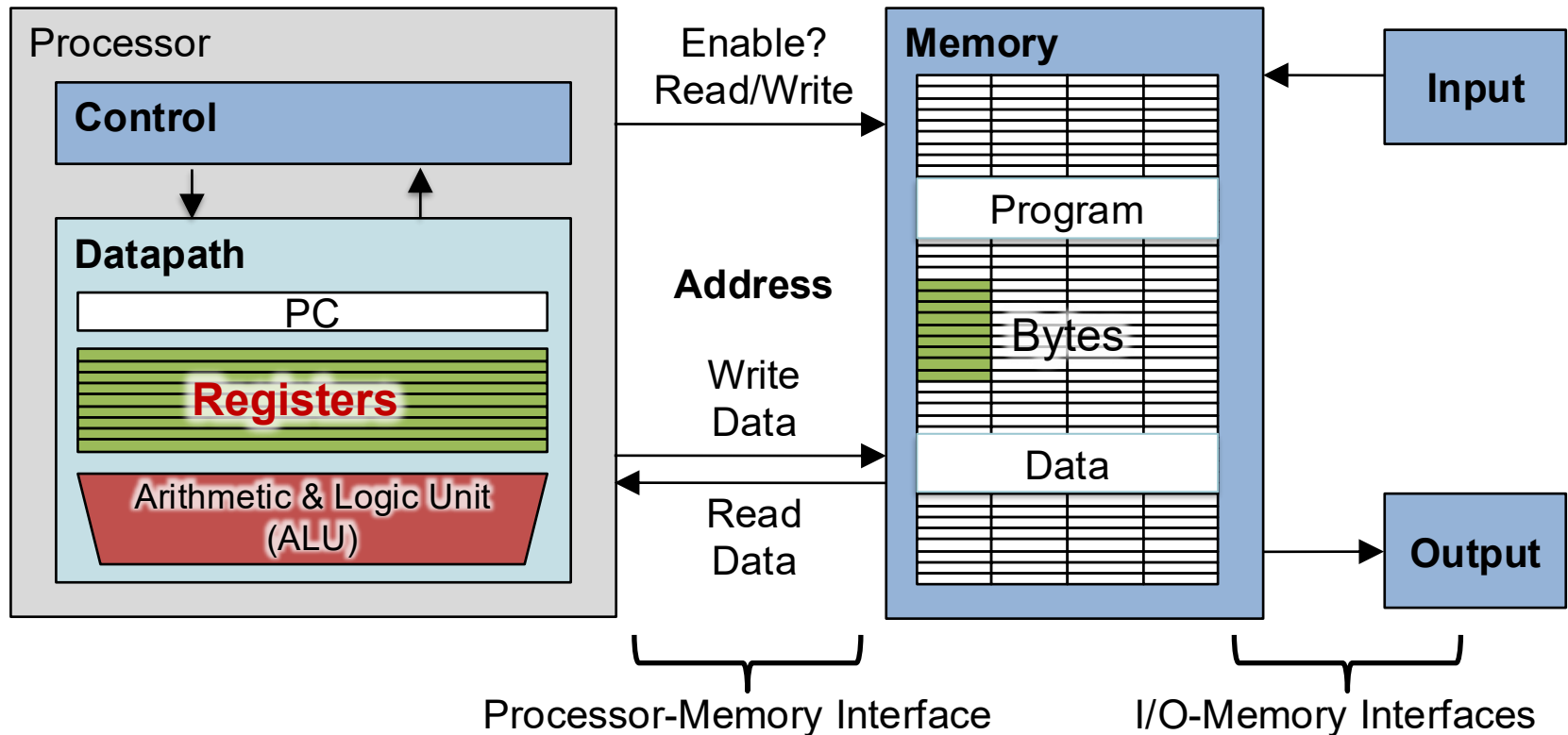
- The processor and memory exchange instructions and data during execution

■ **I/O**

- Connects the computer to the outside world

Basic Computer System

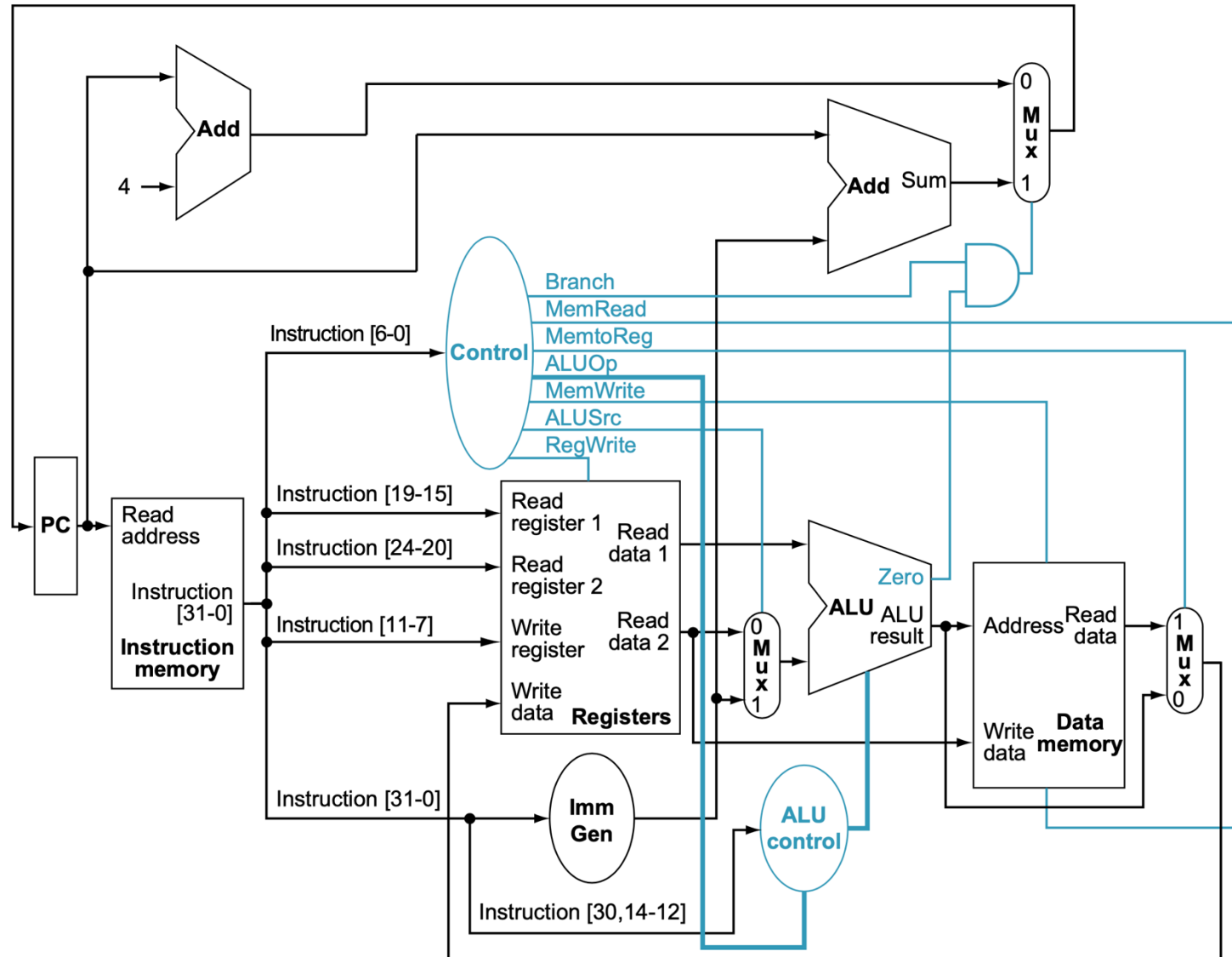
- This is the basic picture of a computer
 - Don't worry about the details yet



How Does The CPU Execute an Instruction?

- **The basic instruction cycle**
 - Fetch → Decode → Execute → Memory access → Write back → Next instruction
- **The processor repeatedly transforms:**
 - Instructions + Machine state → New machine state
- **We will spend much more time understanding this later**

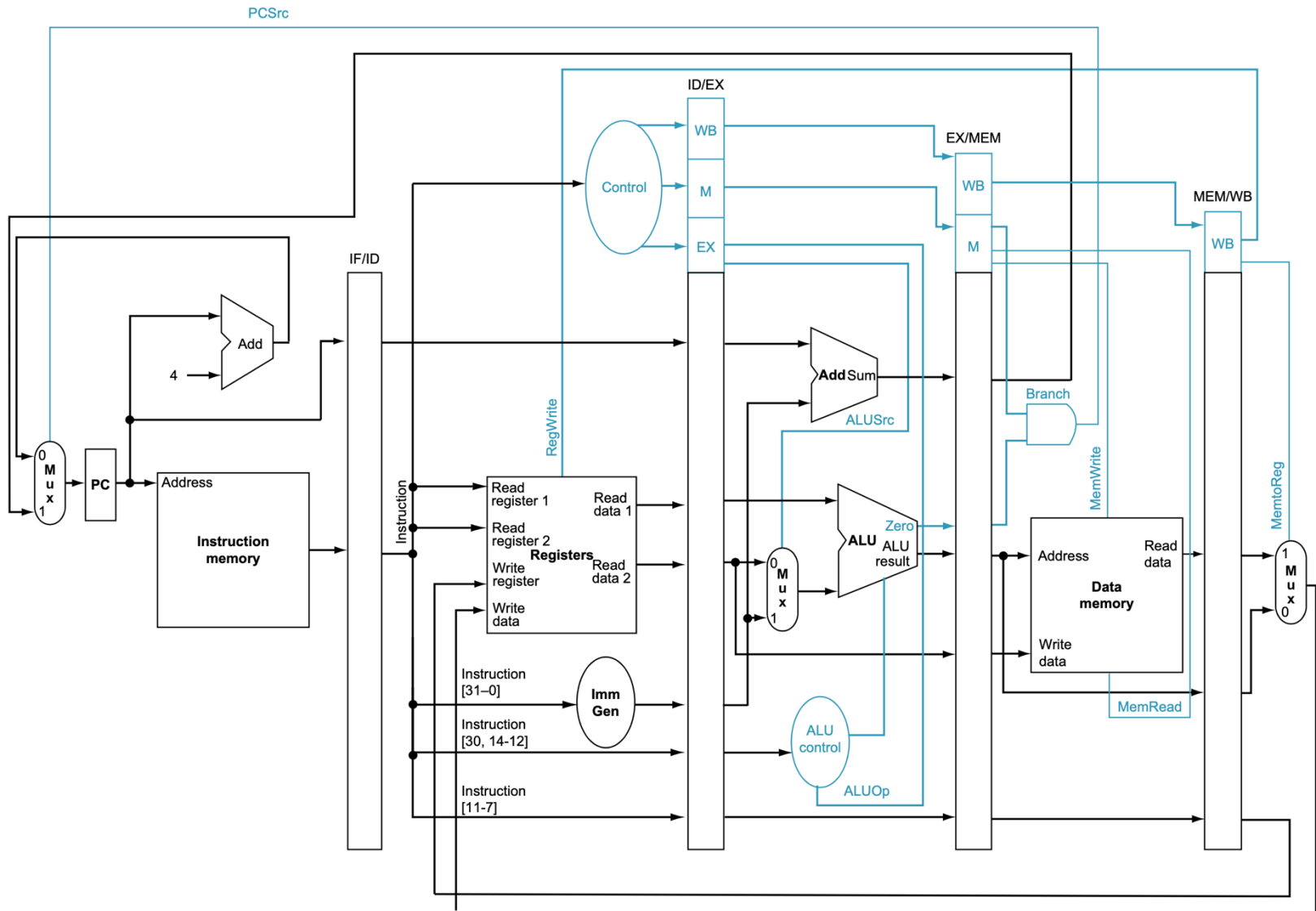
How To Design a Simple Processor?



How Can We Make The Processor Faster?

- A simple processor executes instructions step by step
- **Pipelining** overlaps the execution of multiple instructions
 - Each instruction is divided into several stages
 - Different instructions can be in different stages at the same time
 - Once the pipeline is full, one instruction can complete every clock cycle

A Pipelined CPU



One Processor, Many Programs

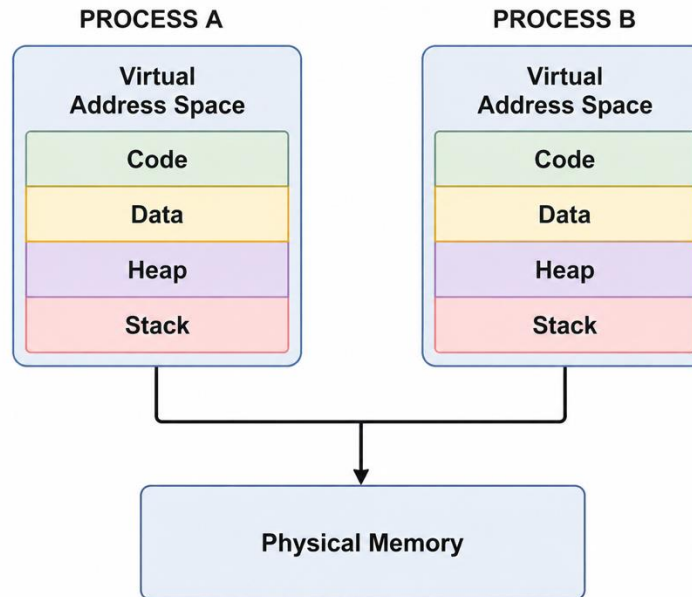
- **So far, we have looked at how one program uses the machine**
 - But a real operating system must make the hardware safely support many programs at once
- **How can many programs share one processor?**
 - We want multiple programs to share the processor and appear to run concurrently

One Processor, Many Programs

- **The operating system manages:**
 - Processes
 - Processor time
 - Registers and execution state
 - Memory
 - I/O resources
- **Context switching**
 - Save one execution state → restore another → continue

Virtual Memory

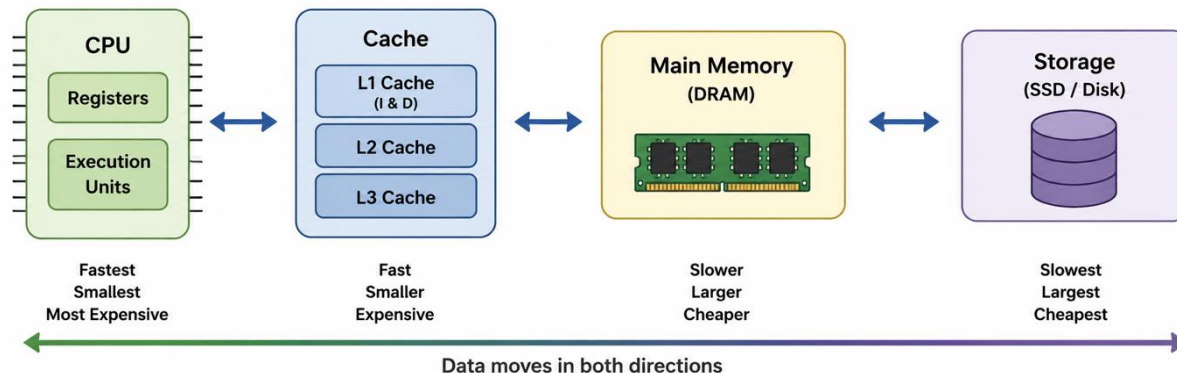
- Each process needs its own view of memory



- Each process gets its own **virtual address space**
 - This provides isolation, protection, independent address spaces, controlled sharing

Why Can't We Use Only Fast Memory?

- **The fundamental trade-off**
 - Fast memory is expensive and limited
 - Large memory is slower and cheaper
- **Key idea**
 - Hierarchy allows us to combine different technologies

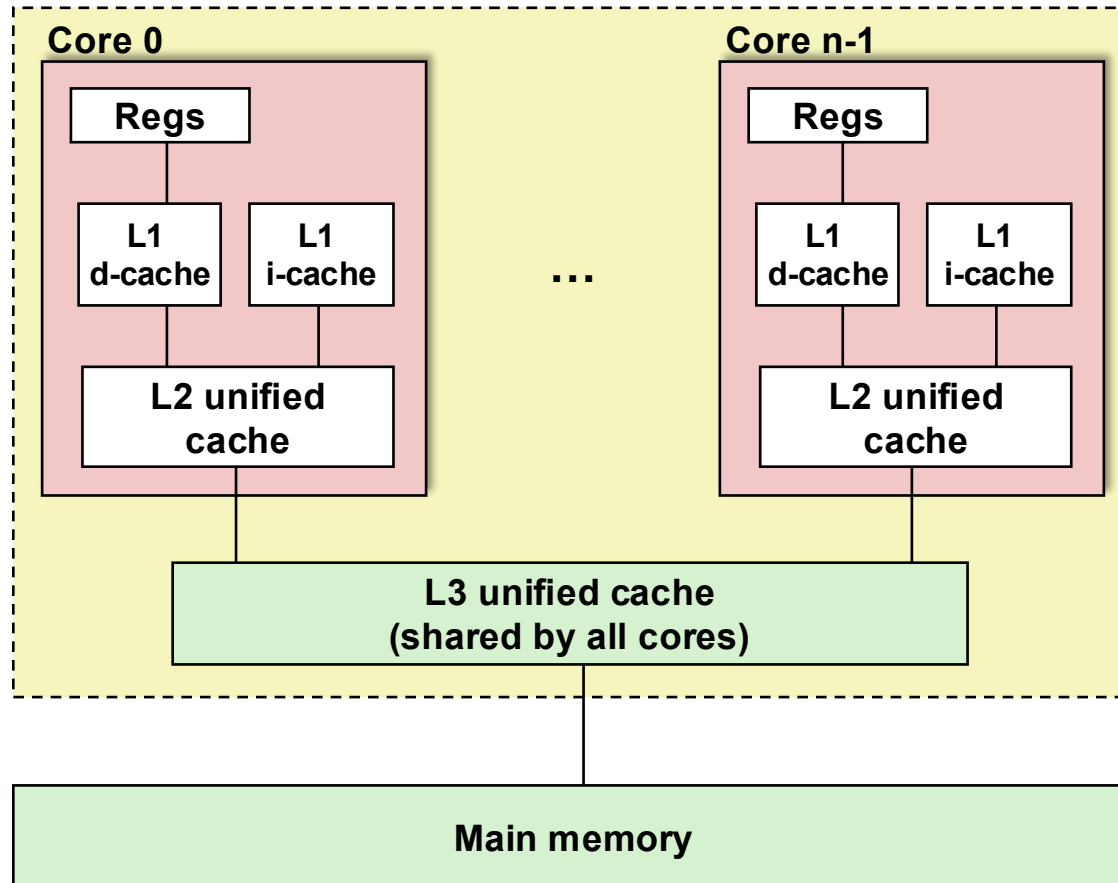


Why Parallelism?

- **Performance cannot rely only on faster clocks**
- **Increasing frequency is limited by:**
 - Power and thermal limits
 - Technology scaling
 - Diminishing returns
- **The alternative is to do more work at the same time**
 - Instruction-level parallelism
 - Data-level parallelism
 - Thread-level parallelism
 - Multicore architectures

Multicore Processors

- More cores, more parallel execution

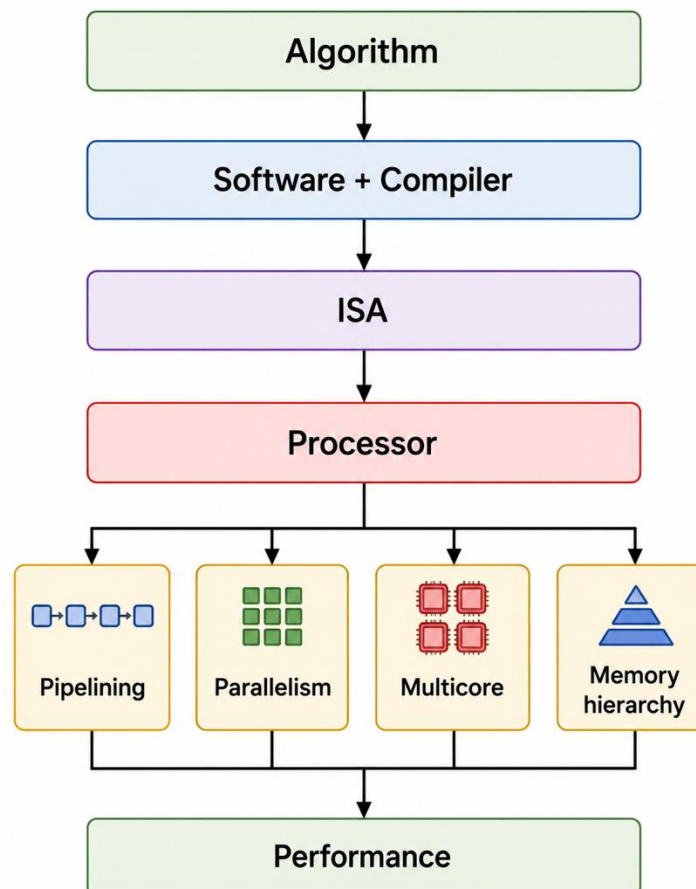


Multicore Processors

- **But 4 cores do not automatically make every program 4x faster!**
 - If 25% of a program is inherently sequential, can 4 cores ever give 4x speedup?
- **Why?**
 - The program must expose parallelism
 - Cores must coordinate
 - Memory can become a bottleneck
 - Some work remains sequential

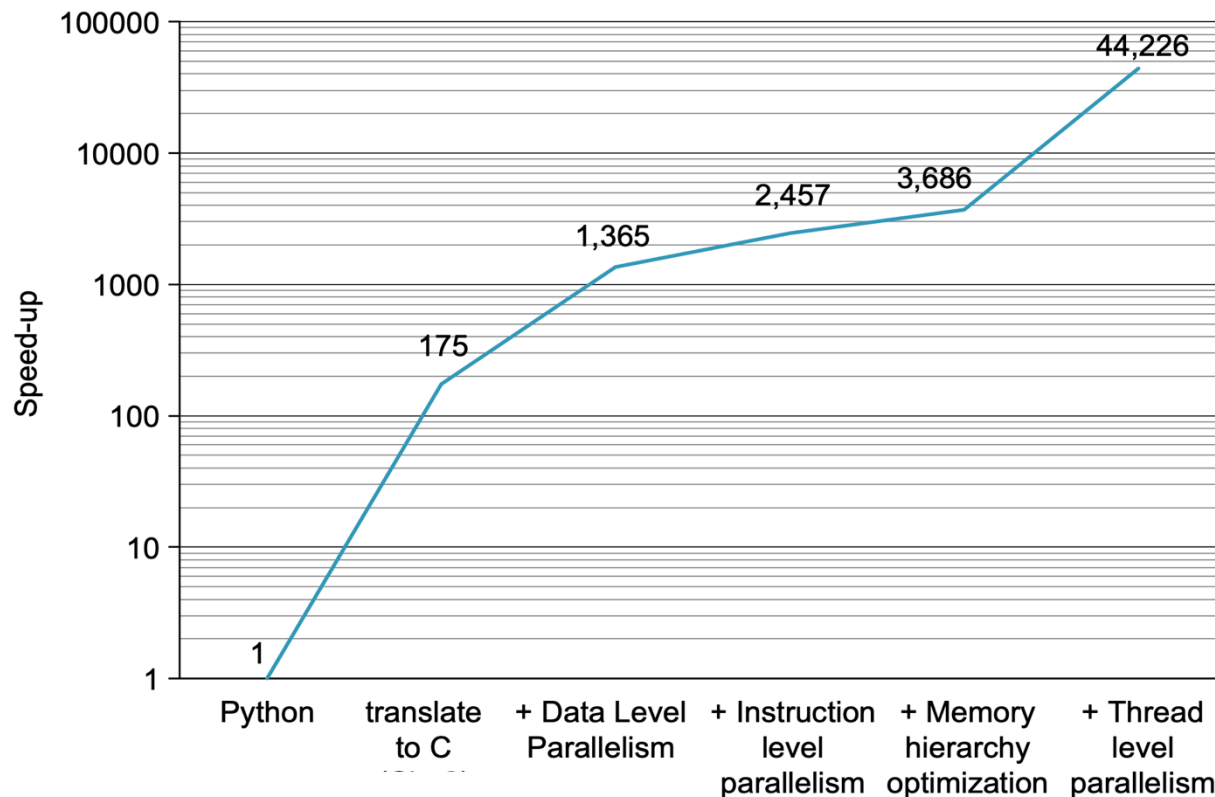
Putting Everything Together

- The same algorithm can run orders of magnitude faster depending on how the system is designed



So, Why Is The Same Program So Much Faster?

- **At the beginning of the lecture, this was just a graph**
 - Now you have seen what these optimizations mean



Before The Next Lecture

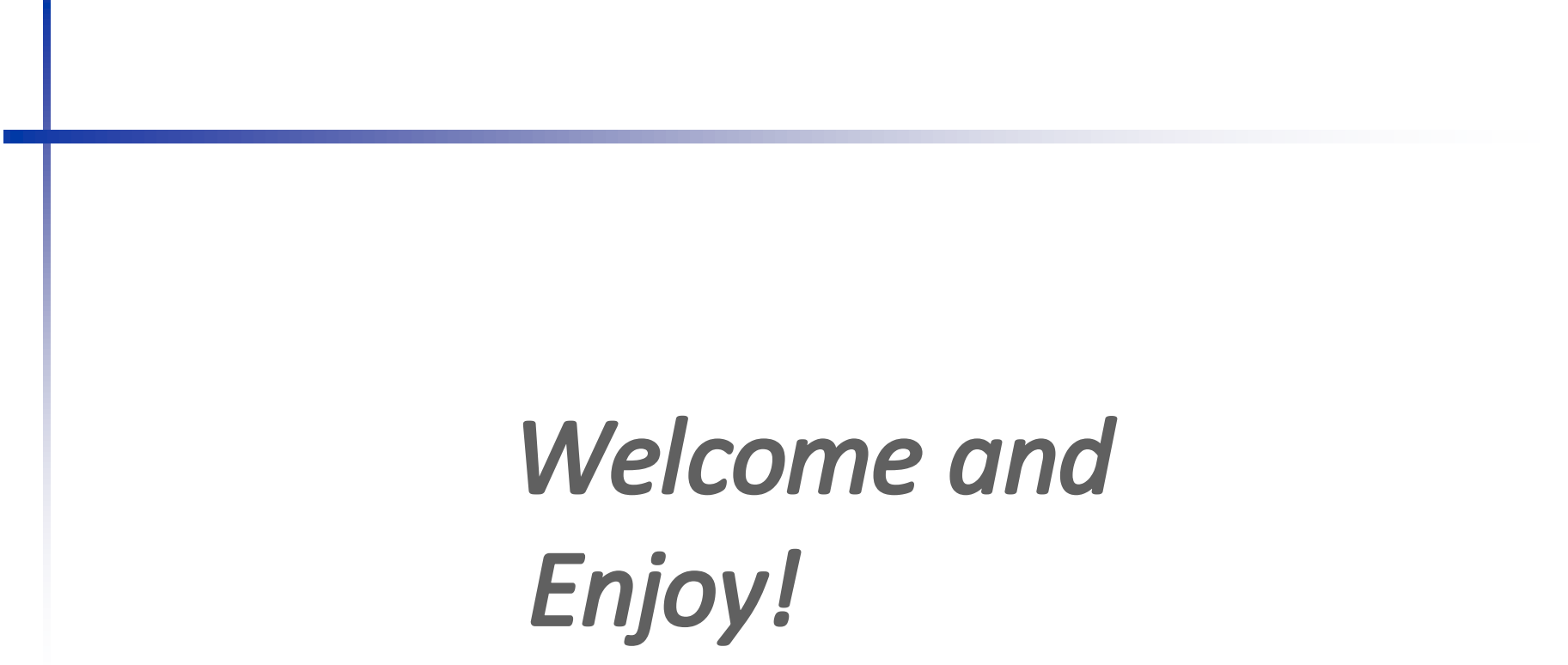
- **Some concepts from your undergraduate courses will be assumed**
- **If you are not comfortable with:**
 - Binary/hexadecimal representation
 - Two's complement
 - Bitwise operations
 - Integer arithmetic
 - Pointers
 - Endianness and memory organization
- **Review the "Data Representation" material before the next class**

Grading

- **Research paper analysis and survey — 40%**
 - Critical evaluation of research papers
 - Research survey
 - Understanding recent developments in critical computer system architectures
- **Final closed-book written exam — 60%**
- **Minimum grade**
 - 7.5/20 in each component

Bibliography

- **“Computer Organization and Design RISC-V Edition: The Hardware/Software Interface, 2nd Edition”**
 - by David Patterson and John Hennessy
- **“Digital Design and Computer Architecture, RISC-V Edition”**
 - by Sarah Harris and David Harris
- **“Computer Organization and Design: A Quantitative Approach, 7th Edition”**
 - by John Hennessy and David Patterson



*Welcome and
Enjoy!*